

ElastoSLM

Nested Low-Rank Elasticity for Small Language Models via Multi-Objective Budget Optimization

Field	Detail
Working title	ElastoSLM: nested low-rank elasticity for SLMs under multi-objective budget allocation
Group / theme	Edge AI — adaptive inference, ML systems, LLM serving
Supervision	Sudaksh Kalra (daily supervisor), University of Amsterdam
Duration	6–9 months, full-time MSc thesis
Prior work built on	Elastoformer (SEC '25), FlexRank (ICML 2026), MOSP (NeurIPS 2025)
Type	Method development + systems evaluation; targets a workshop or conference submission

1. Summary

Small language models in the 0.5–3B range are becoming the practical substrate for on-device assistants, but they are still shipped as fixed-cost monoliths: one checkpoint, one compute footprint, regardless of how much battery, memory, or latency headroom the device happens to have at that moment, and regardless of whether the incoming query is trivial or hard.

This project develops ElastoSLM, a framework that converts a pretrained SLM into a single set of weights from which a family of nested low-rank submodels can be instantiated at deployment time, each satisfying a different compute budget. The technical substrate — low-rank factorization with importance-ordered, nested components — is established. The methodological contribution of this thesis lies elsewhere.

Existing elastic-model training solves the multi-budget problem by **linear scalarization**: a fixed weighted sum of per-budget losses. This is a modelling choice, not a neutral one, and it has a known and provable failure mode — it cannot reach Pareto-optimal configurations that lie in non-convex regions of the front, for *any* choice of weights. The student will reformulate multi-budget elastic training as an explicit multi-objective optimization (MOO) problem, following the multi-objective framing that Chen et al. (NeurIPS 2025) introduced for one-shot LLM pruning, and then extend it to a two-dimensional nesting over both compute budget and task capability — a structure that, to our knowledge, no existing method provides.

2. Background and Motivation

2.1 Two axes of elasticity

Work on elastic neural networks has converged on a common recipe — one shared parameter set, many extractable submodels — but splits on which axis the submodels are nested along.

Method	Nests along	Mechanism
Elastoformer (SEC '25)	Compute budget	Iterative structured pruning (compress) followed by weight-sharing regrowth with a

Method	Nests along	Mechanism
		frozen prefix; runtime mode selection from a profiled lookup table; measured on Jetson Orin/Nano
FlexRank (ICML 2026)	Compute budget	Per-layer DataSVD giving importance-ordered rank components; dynamic-programming search for per-layer rank profiles; nested submodel training via distillation from the base model
MOSP (NeurIPS 2025)	Task capability	One-shot pruning as MOO over per-task calibration sets; dual ADMM yields a core support nested inside task-specific supports; a preference vector λ selects a Pareto point at inference

Elastoformer and FlexRank both answer "how much compute do I have?". MOSP answers "which capability do I care about?". Neither answers both, and the mechanisms are complementary rather than competing — one nests rank components, the other nests weight supports.

2.2 The gap this thesis attacks

FlexRank's elastic training objective, over K budget levels with per-budget distillation losses $\ell_1 \dots \ell_K$, is:

$$\min_{\theta} \sum_k \alpha_k \ell_k(\theta) \quad \text{subject to} \quad \sum_k \alpha_k = 1, \quad \alpha_k > 0$$

This is a linear scalarization of the vector-valued objective $(\ell_1(\theta), \dots, \ell_K(\theta))$. Two consequences follow directly from standard multi-objective optimization theory:

1. **Unreachable solutions.** Weighted-sum scalarization can only recover Pareto points lying on the convex hull of the front. Points in non-convex (concave) regions are unattainable for every possible α . No amount of tuning fixes this — it is structural.
2. **Poor coverage.** Even where the front is convex, the map from α to the resulting Pareto point is highly non-uniform, so a uniform α (the default in the released FlexRank scripts) gives no control over how the recovered solutions are distributed across budgets.

This matters specifically for elastic models because the budgets that are practically interesting are the aggressive ones, and that is precisely where the accuracy-versus-budget curve is most likely to bend the wrong way. FlexRank's own Llama-3.2-1B results degrade sharply below roughly 40% of parameters — exactly the regime where a non-convex kink would hide Pareto points from the scalarized objective.

MOSP provides both the precedent and a cautionary result. Its Table 1 shows that naive multi-objective extensions of a strong single-objective pruner collapse: activation concatenation gives 8.72 average perplexity and independent-prune-then-merge gives 142.51, against 7.68 for per-task pruning, on LLaMA-2-7B at 70% sparsity. Treating multiple objectives properly is not a cosmetic change.

The opportunity: MOSP's objectives are capabilities and its mechanism is sparsity. FlexRank's objectives are budgets and its mechanism is rank. ElastoSLM applies MOSP's *formulation* to FlexRank's *axis*, then unifies the two into a single model indexed by a budget β and a preference λ .

3. Research Questions

Each question is stated so that a negative answer is still a reportable result.

ID	Question	How it is settled
RQ1	Does weighted-sum scalarization leave Pareto-optimal budget configurations unreachable at SLM scale, and does gradient-based MOO recover them?	Hypervolume of the recovered front vs an α -sweep baseline at matched compute; explicit search for non-convex regions in the budget-loss curve
RQ2	Which MOO family is appropriate when the objectives are compute budgets sharing nested weights: common-descent methods or preference-conditioned models?	Head-to-head on hypervolume, ϵ -indicator and front spread, with wall-clock training overhead reported
RQ3	Can budget nesting and capability nesting coexist in one weight set, or do they interfere?	A (β, λ) -indexed model evaluated against single- λ elastic models at matched β on general / math / code
RQ4	Do parameter reductions translate into wall-clock gains for SLM decoding at batch size 1 on edge hardware?	TTFT and TPOT reported separately on Jetson Orin; profile-switch latency measured
RQ5	Does the additive-error assumption underpinning the DP profile search hold at 1-3B scale?	Rank correlation between DP-predicted and measured error over sampled profiles

RQ1 is the pivot. If the scalarization gap is real, the thesis has a clean methodological result. If it is empirically negligible at this scale, that is itself worth publishing — it would tell the elastic-models community that the simple objective is sufficient, and the project pivots its emphasis to RQ3 and RQ4.

4. Proposed Approach

4.1 Stage A — a nested low-rank elastic SLM

Each linear projection $W \in \mathbb{R}^{(m \times n)}$ is factorized as $W \approx U V^T$ using an activation-aware decomposition that minimizes output reconstruction error $\mathbb{E} \| (W - UV^T)x \|^2$ over a calibration set, rather than weight error. Budget level k is the rank prefix $U[:, :r_k]$, $V[:, :r_k]$, so nestedness is a contiguous slice and mode switching is one integer per layer.

- **Architectural constraints.** Rank factorization preserves `head_dim` exactly, so RoPE's $(2i, 2i+1)$ pairing and GQA key/value head grouping survive untouched. This is the decisive advantage over head-dimension pruning, which breaks both — and it is why the rank axis is the right starting point for language models specifically.
- **Scope of decomposition.** Per-layer projections only. Embeddings and the LM head are excluded: they are a large parameter fraction at SLM scale, and any reported compression ratio must state explicitly whether they are counted. The residual stream width is never altered.
- **Capped maximum rank.** Full-rank factors cost $2mn$ — a $2\times$ memory penalty that would defeat the purpose. Storing only up to $r_{\max}$ gives $(m+n) \cdot r_{\max}$; at $r_{\max} = 0.3d$ for a square layer that is $0.6d^2$, smaller than the dense d^2 . The model is then below the base model's footprint at every mode, at the cost of the top mode no longer reproducing the pretrained network exactly. Quantifying that trade is part of Stage A.
- **Profile search and consolidation.** Dynamic-programming allocation of ranks across layers under a global budget, then nested submodel training distilled from the frozen base model, which is available for free and provides a far richer signal than labels.

4.2 Stage B — multi-objective budget optimization (core contribution)

Replace the scalarized objective with the vector-valued problem

$$\min_{\theta} (\ell_1(\theta), \ell_2(\theta), \dots, \ell_K(\theta)) \quad \text{subject to nestedness } m_1 \preceq m_2 \preceq \dots \preceq m_K$$

and pursue two tracks, to be compared directly:

B1 — Common-descent methods. At each step, compute per-budget gradients and combine them into an update direction that does not increase any ℓ_k , via MGDA, CAGrad, or Nash-MTL. The obvious cost is K backward passes per step; mitigation is stochastic subsampling of two or three budgets per step combined with an upper-bound approximation of the multiple-gradient descent direction. Characterising this overhead honestly is part of the deliverable.

B2 — Preference-conditioned models. Learn the whole front rather than K points: a small hypernetwork conditioned on the budget produces per-layer rank allocations and/or modulation coefficients for the shared factors, in the spirit of Pareto-front hypernetworks. Attractive because it turns the DP search from a fixed preprocessing step into a learned, continuously interpolable policy — and because it makes input-dependent budget selection a natural downstream extension.

Both are measured against the scalarization baseline at matched compute. The FlexRank best-submodel optimality gap provides a second, theory-grounded diagnostic alongside hypervolume.

4.3 Stage C — two-axis nesting over budget and capability

MOSP identifies a core weight support shared across tasks, nested inside task-specific supports, and blends task-specific corrections at inference according to a user preference λ . Stage C ports this idea into rank space: a core rank block shared across all capabilities, plus capability-specific rank extensions. Given a pair (β, λ) , the deployed weights are the core prefix plus a λ -weighted combination of capability extensions, truncated to the budget β .

Calibration sets follow MOSP's structure — general text, mathematical reasoning, and code — drawn from FineWeb-Edu, GSM8K-style, and Stack/MBPP-style data respectively.

Two open questions the thesis must resolve: (i) whether the λ -blend of capability extensions remains a valid factorization or requires re-orthogonalization, and (ii) whether the inference-time reparametrization that makes rank truncation fast survives the blend. Both are genuine unknowns and should be treated as findings, not obstacles.

5. Work Packages and Timeline

Nine months, with an explicit go/no-go gate at the end of each package. The gates exist so that a stalled package converts into a scoped negative result rather than an open-ended delay.

WP	Months	Content	Gate
WP0	M1	Literature review; environment setup; reproduce the published elastic low-rank Pareto curve on GPT-2 and Llama-3.2-1B	Published curve reproduced within tolerance
WP1	M2–M3	Stage A: activation-aware decomposition, capped $r_{\max}$, DP profile search, nested distillation, on two SLMs	Elastic front beats the decomposition-plus-search baseline without consolidation
WP2	M4–M6	Stage B: MOO reformulation; implement and compare B1 and B2 against scalarization; answer RQ1 and RQ5	Hypervolume improvement at matched compute, or a documented negative result for RQ1

WP	Months	Content	Gate
WP3	M6–M8	Stage C: two-axis (β , λ) nesting; per-capability calibration; blend validity and reparametrization study	(β , λ) model beats single- λ elastic at matched β on ≥ 2 of 3 capabilities
WP4	M8–M9	Jetson Orin deployment: TTFT/TPOT at batch 1, switch latency, KV-cache reuse across profiles; thesis and paper draft	Thesis submitted; systems results support or refute RQ4

Note on ordering. WP2 begins before WP3 finishes deliberately: Stage B is the core contribution and must have the most slack. If WP3 runs long it can be truncated to a single-capability demonstration without endangering the thesis.

6. Evaluation Protocol

6.1 Models

Llama-3.2-1B, Qwen3-1.7B, and one 3B-class model (SmolLM3 or Gemma-3-1B/4B), base checkpoints, with instruction-tuned variants if time permits. The 0.5–3B range is chosen deliberately: it is the band that is actually deployable on edge accelerators, and it is trainable with distillation on an 8×RTX 3090 class cluster.

A secondary question worth a subsection: Llama-3.2-1B is itself distilled from a larger model. Does a distilled SLM have less exploitable redundancy — and therefore worse rank-elasticity — than a natively pretrained model of the same size?

6.2 Metrics

Family	Metrics
Quality	Held-out perplexity (FineWeb-Edu); lm-eval-harness commonsense suite; GSM8K for mathematical reasoning; MBPP/HumanEval for code; IFEval on instruction-tuned checkpoints
Multi-objective	Hypervolume; ϵ -indicator; spread and coverage of the recovered front; best-submodel optimality gap
Systems	Parameter count (with and without embeddings); TTFT and TPOT reported separately; peak memory; profile-switch latency; measured on both datacentre GPUs and Jetson Orin at batch size 1

On TTFT versus TPOT. Prefill is compute-bound and sees only the FLOP reduction; decode is weight-bandwidth-bound, so a parameter reduction translates almost one-to-one into per-token latency. Reporting a single averaged latency number conflates two different regimes and hides the result that most favours low-rank compression on edge devices. Neither of the parent papers separates them.

6.3 Baselines

- Truncated SVD and activation-aware SVD with DP search, without knowledge consolidation — isolates the effect of nested training
- FlexRank with uniform α — the primary baseline, and the direct target of RQ1
- ACIP, the current state of the art in low-rank elastic compression

- LLM-Pruner (structured sparsity) and LayerSkip (depth elasticity) for cross-family context
- MOSP as the multi-objective reference point, on the sparsity axis
- Independently trained submodels at matched total compute — the non-elastic upper bound

7. Expected Contributions

3. **A formulation** of multi-budget elastic training as an explicit multi-objective optimization problem, with a characterization of when and how far weighted-sum scalarization falls short at SLM scale.
4. **ElastoSLM**, an open-source nested low-rank elastic framework for small language models, with a capped-rank storage scheme that is smaller than the dense base model at every operating mode.
5. **A two-axis elastic model** indexed jointly by compute budget and capability preference — a structure not provided by any existing method.
6. **Edge measurements** that separate prefill from decode and report profile-switch cost, filling a gap the low-rank compression literature currently leaves open.
7. **An audit** of the additive-error assumption underlying dynamic-programming rank allocation, at a scale two orders of magnitude beyond where it was originally validated.

8. Risks and Mitigations

Risk	Impact	Mitigation
MOO gradient methods require one backward pass per objective, making training $K\times$ more expensive	Blocks WP2 at realistic model scale	Stochastic subsampling of 2–3 budgets per step; upper-bound approximations; fall back to track B2, whose cost is independent of K
Low-rank FLOP savings fail to materialize as wall-clock gains at batch size 1	Undermines the deployment story	Report the result honestly (it is publishable); fall back to a hybrid that keeps low rank only where the DP requests aggressive reduction and uses permuted structured pruning elsewhere, which preserves dense kernels
The Pareto front turns out to be convex, so RQ1 returns negative	Removes the headline result	Pre-planned pivot: the thesis re-centres on RQ3 (two-axis nesting) and RQ4; the negative result is reported as a finding about elastic training objectives
Compute limits distillation on 3B-class models	Reduces model coverage	SLM range chosen for this reason; gradient checkpointing and bf16; 3B treated as a stretch model, 1B as the primary
λ -blending of capability extensions breaks the factorization or the fast inference form	Weakens Stage C	Investigate re-orthogonalization; failing that, store per-capability profiles and switch discretely rather than blending continuously
Scope creep across three stages in nine months	Late or unfinished thesis	Hard go/no-go gates per WP; WP3 is explicitly truncatable to a single capability

9. Prerequisites and Supervision

The project suits a student who is comfortable reading a method paper and reimplementing it, rather than one looking for a well-specified engineering task.

- Strong PyTorch; familiarity with Hugging Face transformers and accelerate, or willingness to acquire it in month one
- Solid linear algebra — SVD, low-rank approximation, and subspace reasoning are load-bearing throughout, not incidental
- Some background in optimization; prior exposure to multi-objective or multi-task optimization is an advantage but can be picked up during WP2
- Comfort with multi-GPU cluster workflows and experiment tracking
- Useful but not required: experience with Jetson-class hardware, quantization, or inference-serving stacks

Supervision is weekly, with a written progress note before each meeting and a formal review at each work-package gate. The student is expected to keep a reproducible experiment log from week one; the Pareto-front results are the thesis, and they must be regenerable.

10. Resources

- Compute: 8×NVIDIA RTX 3090 (24 GB) for training and consolidation; an NVIDIA Jetson Orin for deployment measurement in WP4
- Codebases: Elastoformer (structured-pruning elasticity, edge measurement harness); FlexRank core package (decomposition, DP search, samplers, trainers), which is deliberately standalone and can be used as the compression backend
- Data: FineWeb-Edu for consolidation and general calibration; GSM8K-style and Stack/MBPP-style corpora for the capability axis in Stage C
- Evaluation: lm-eval-harness for downstream benchmarking

11. Key References

- [1] Kalra, S. and Sapra, D. Elastoformer: Enabling Dynamic Adaptivity via Elastic Model Transformation. ACM/IEEE Symposium on Edge Computing (SEC '25), 2025.
- [2] Zacccone, R., Laskaridis, S., Ciccone, M. and Horváth, S. FlexRank: Nested Low-Rank Knowledge Decomposition for Adaptive Model Deployment. ICML 2026.
- [3] Chen, W., Yang, H., Gou, Y., Shi, H., Hu, E., Li, Z. and Kwok, J. T. Multi-Objective One-Shot Pruning for Large Language Models. NeurIPS 2025.
- [4] Devvrit, F., Kudugunta, S., Kusupati, A. et al. MatFormer: Nested Transformer for Elastic Inference. NeurIPS 2024.
- [5] Genzel, M. et al. ACIP: Attention-aware Compression with Iterative Pruning for elastic low-rank LLMs.
- [6] Ma, X., Fang, G. and Wang, X. LLM-Pruner: On the Structural Pruning of Large Language Models. NeurIPS 2023.
- [7] Elhoushi, M. et al. LayerSkip: Enabling Early Exit Inference and Self-Speculative Decoding. ACL 2024.
- [8] Sun, M., Liu, Z., Bair, A. and Kolter, J. Z. A Simple and Effective Pruning Approach for Large Language Models (Wanda). ICLR 2024.
- [9] Frantar, E. and Alistarh, D. SparseGPT: Massive Language Models Can Be Accurately Pruned in One-Shot. ICML 2023.
- [10] Sener, O. and Koltun, V. Multi-Task Learning as Multi-Objective Optimization (MGDA). NeurIPS 2018.

- [11] Liu, B., Liu, X., Jin, X., Stone, P. and Liu, Q. Conflict-Averse Gradient Descent for Multi-task Learning (CAGrad). NeurIPS 2021.
- [12] Navon, A., Shamsian, A., Achituve, I. et al. Multi-Task Learning as a Bargaining Game (Nash-MTL). ICML 2022.
- [13] Navon, A., Shamsian, A., Chechik, G. and Fetaya, E. Learning the Pareto Front with Hypernetworks. ICLR 2021.
- [14] Yu, J., Yang, L., Xu, N., Yang, J. and Huang, T. Slimmable Neural Networks. ICLR 2019.
- [15] Cai, H., Gan, C., Wang, T., Zhang, Z. and Han, S. Once-for-All: Train One Network and Specialize It for Efficient Deployment. ICLR 2020.
- [16] Chen, P., Yu, H.-F., Dhillon, I. and Hsieh, C.-J. DRONE: Data-aware Low-rank Compression for Large NLP Models. NeurIPS 2021.